

Connecting the Dots: Automated Strategies for Semantic Grouping to Solve NYT Connections

By: David Zhang

Csc 447: Introduction to Machine Learning

Prof. Erik Grimmerman

May 16, 2026

Abstract

Natural language processing is a well-researched topic in machine learning, and this project looks into automated machine learning strategies for semantic grouping by solving the NYT game Connections. Connections is a game of categorizing sixteen distinct words into four specific groups based on varying levels of semantic, structural, and abstract relationships. To solve this game, three methodologies were developed and evaluated: static embeddings, a siamese network, and existing large language models. The static embeddings approach combined GloVe, WordNet, and JellyFish datasets into weighted affinity matrices navigated via a greedy depth-first-search on valid groupings, achieving a 28.1% solve rate across historical data^{[1][2][3]}. To capture more nuanced, non-literal wordplay, a siamese network was trained to warp the vector space of an existing sentence transformer (MiniLM-L6-v2)⁴ and utilized a heuristic lookahead beam search, yielding a 34% solve rate. The last method was using existing LLMs like ChatGPT, Gemini, and Claude. Unsurprisingly, this vastly outperforms the automated methods and even human baselines with near-perfect accuracy. The results highlight the limitations of rigid vector distances and static lexical databases when faced with polysemy, anagrams, and niche cultural contexts. Ultimately, the project demonstrates that successfully automating complex semantic grouping requires dynamic contextual reasoning that extends far beyond traditional vector semantics.

¹ Pennington, Jeffrey, et al. *Glove: Global Vectors for Word Representation*, nlp.stanford.edu/projects/glove/. Accessed 12 Apr. 2026.

² “WordNet.” *Princeton University*, The Trustees of Princeton University, wordnet.princeton.edu/. Accessed 12 Apr. 2026.

³ Jamesturk. “Jamesturk/Jellyfish: 🐙 a Python Library for Doing Approximate and Phonetic Matching of Strings.” *GitHub*, github.com/jamesturk/jellyfish. Accessed 11 Apr. 2026.

⁴ “Sentence-Transformers/All-MiniLM-L6-V2 · Hugging Face.” *Sentence-Transformers/All-MiniLM-L6-v2 · Hugging Face*, huggingface.co/sentence-transformers/all-MiniLM-L6-v2. Accessed 21 May 2026.

1.Introduction

The rapid advancement of Natural Language Processing has continuously shifted the benchmarks used to evaluate machine intelligence. Traditional semantic tasks often rely on direct synonym matching, hypernym identification, or simple keyword proximity. However, human-centric language puzzles introduce complexities that test the absolute boundaries of vector space semantics. An example is the popular New York Times daily puzzle game, Connections. This project explores different ways of solving this semantic puzzle with machine learning.

1.1 What is Connections

The core of this game is to sort a discrete board of sixteen unique words and partition them into four groups of four. The game structure is based on four distinct semantic categories of increasing difficulty:

- Straightforward (Level: Easy): Categories typically based on highly literal definitions, direct synonyms, or functional groupings.
- Common Groups (Level: Medium): Categories that require a slightly broader context, such as collective nouns, shared consumer brands, or adjacent semantic spaces.
- Nuanced Association (Level: Hard): Groupings that introduce linguistic misdirection, idioms, or slight wordplay and trivia.
- Abstract Connection (Level: Tricky): Highly creative or structural categories, frequently relying on wordplay, homophones, anagrams, or niche cultural knowledge.

1.2 The math behind connections

Mathematically, let the puzzle board be represented as a set 'W' containing exactly sixteen unique words:

$$W = \{w_1, w_2, \dots, w_{16}\}$$

The computational objective is to resolve a definitive partition of ‘W’ into four disjoint subsets, or groups, G_1, G_2, G_3, G_4 , mapping difficulty tiers such that:

$$\forall i \in \{1, 2, 3, 4\}, \quad |G_i| = 4 \quad \text{and} \quad \bigcup_{i=1}^4 G_i = W$$

The problem space is deceptively vast; a simple combinatorial selection yields, 16 choose 4, 1,820 unique ways to form a single four-word group. When evaluating the total valid combinations required to completely partition the entire 16-word board into four sequential groups, the problem expands exponentially to a total search space of, $16! / (4!)^4$, 2,627,625 possible combinations.

1.3 Adversarial Linguistics

What makes Connections a test of NLP capability is the intentional presence of deceptive overlap, or "red herrings". The game is made to ensure that individual words can fit within multiple intuitive categories. For instance, a single board might contain five or six words that closely associate with medical professions (e.g., Nurse, Doctor, Surgeon), but a globally optimal solution forces the extraction of specific elements to satisfy a completely separate, hidden linguistic pattern.

Consequently, a local optimization strategy that greedily clusters the most obvious semantic cohorts will inevitably stall, exhausting the maximum permitted mistake threshold before resolving the global partition. Solving this requires going past pure dictionary definitions and modeling the implicit fluid mechanics of human wordplay. Natural language resists mapping onto rigid vector distances due to the polysemy, where words may possess identical structures

but diverge across definitions, contextual implementations, and phonetics. When an abstract category introduces patterns built on structural modifications (such as dropping letters) or cultural trivia, vector-based semantic maps lose their signal completely.

2. Project Overview

The primary objective of this project is to design, implement, and benchmark an automated framework capable of solving the structural and adversarial linguistic challenges found in the game Connections. The computational system is engineered to ingest a sixteen-word board matrix and partition it into four distinct, disjoint subsets of four words each. Because the game is intentionally designed with deceptive overlaps and red herrings, a standard clustering model will easily fail. To evaluate how different levels of inference handle this complexity, we will compare three different methods on how accurately they can solve the puzzle.

3. Static Embeddings

To create the similarity matrix for inference, we first need data vast enough to cover the semantic possibilities of categories. For this approach, three sources were used: GloVe (global vectors for word representation), WordNet, and JellyFish. GloVe creates metrics based on a word's co-occurrence with other words. WordNet is a lexical database that creates a metric through a word's definition and synonyms. JellyFish is a Python library that creates a metric for a word's morphology, like its suffix, prefix, and overall spelling.

3.1 The correlation matrix

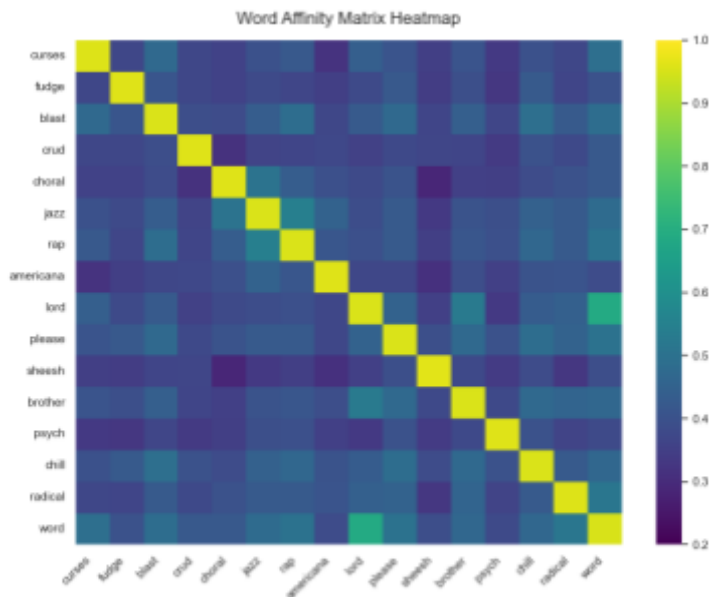


Fig 1. Similarity matrix Heatmap

To get the final similarity matrix for the final inference search, each library first outputs its own matrix that is then combined. But what are the optimal weights for each source to maximize accuracy? Through iterating multiple possible weights, it was found that giving sixty percent to GloVe, thirty-five percent to WordNet, and five percent to JellyFish was the most optimal combination.

3.2 Matrix Search

For the matrix search, a recursive DFS is performed with a prioritized candidate array that greedily selects the highest-scoring cluster to assign a partition. To maintain disjoint groups, any candidates containing those words are pruned from the pool. The search then recursively searches for subsequent combinations. If no valid clusters remain or similarity thresholds are not met, the algorithm backtracks, restoring pruned groups and attempting the next highest-scoring alternative.

Although the search is computationally efficient, there is a possibility that the search will return a local maximum score rather than the global maximum, which may be the correct answer.

3.3 Static Embeddings Results

So how does the approach perform? Over 915 games, it solved 215 of those games with only 93 of those being perfect wins with no mistakes.

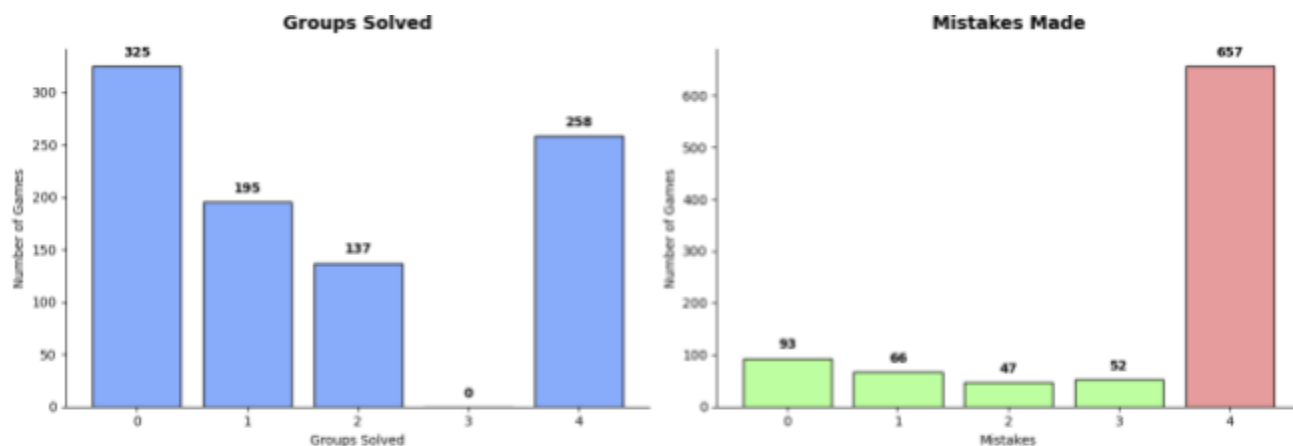


Fig 2. Static Embeddings performance stats



Fig 3. Static Embeddings win percentage

This method resulted in a win rate of 28.1 percent, highlighting the challenges in managing adversarial linguistics. The model averaged 3.2 mistakes per game, often becoming trapped by intentional "red herrings" that align closely in vector space but do not constitute the global solution.

4. Siamese Network

While static embeddings establish baseline linguistic relationships, they struggle to model language with a limited, specialized scope. To move beyond pure semantic grouping, a

secondary model explicitly trained to map the specific types of wordplay found in Connections was needed. Traditional models often overlook the idiosyncratic associations prevalent in the game, necessitating a more dynamic approach to capturing non-literal relationships.

4.1 Model Architecture

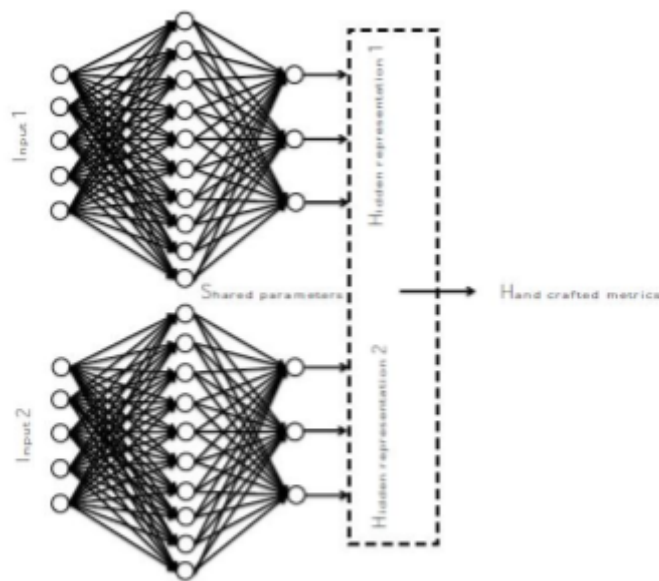


Fig 4. Siamese Network

This methodology uses a Siamese Network, which uses two identical subnet weights to map words into valid clusters.⁵ By feeding the model 1,000 past Connections games, the system successfully created a vector space representing typical gameplay. This newly established vector space then acted as a space warper for an existing word transformer (MiniLM-L6-v2). This architecture enables the network to learn a

distance metric that specifically prioritizes the linguistic patterns unique to the dataset, affecting the created weights and allowing the system to capture clusterings that aren't strictly connected by formal word definitions.

4.2 Lookahead Beam Search

To optimize accuracy instead of using a greedy search, a heuristic beam search can be implemented instead to find the global maximum. Unlike greedy approaches that may become

⁵ "Siamese Neural Network." *Wikipedia*, Wikimedia Foundation, 30 Jan. 2026, en.wikipedia.org/wiki/Siamese_neural_network. Accessed 13 May 2026.

trapped in local optima by selecting the most immediate high-scoring match, the beam search maintains a set of high-probability paths, allowing for more robust navigation of adversarial linguistic overlaps.

The search identifies the top 40 scoring groups using average pairwise cosine distances. It then checks the next available highest-scoring group from that top 40, discarding the lower scores continuously until a valid grouping is successfully found. This lookahead mechanism ensures that the model considers the downstream impact of each grouping decision, effectively minimizing the risk of exhausting mistake thresholds on deceptive "red herrings".

4.3 Siamese Results



Fig 5. Siamese Training Performance

From figure 4, the training wasn't a total success. The model started to show clear signs of overfitting after 15 epochs in training and never managed to capture the structure of wordplay necessary. Despite this it still got a 34 percent solve rate in testing.

5. The LLM

All these methods were ways to mimic how a language model would process word clustering. Now how would a LLM perform with extensive training from texts encompassing the entire internet? Testing three models (Gemini 3.1, ChatGPT-5.5, and Claude-sonet 4.6), the solve rate is vastly greater than the other methods.

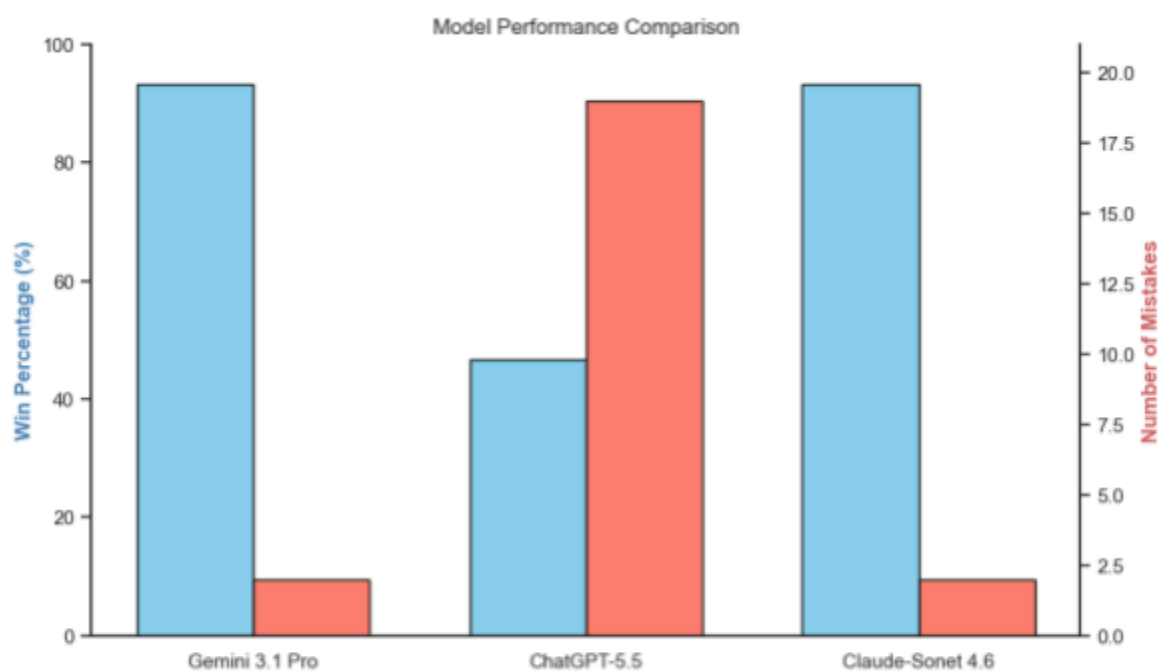


Fig 6. LLM performance

6. Results and Discussion

The evaluation of the three automated approaches reveals a clear hierarchy in their ability to resolve complex semantic groupings. In testing, the Static Embeddings method achieved a solve rate of only 28.1% across 915 archived games. Because the system relies on static datasets like GloVe and WordNet, it has no mathematical representation for categories built on anagrams, fill-in-the-blank phrases, or obscure cultural trivia, causing the greedy search algorithm to fail

entirely on abstract groupings. The Siamese Network demonstrated a measurable improvement, achieving a 34% solve rate. To fully map the complexities of the game's categories, the entire available dataset of 915 archived *Connections* games was utilized strictly as training data without any data splitting. By leveraging this complete dataset to actively warp the latent space of the transformer, the model successfully pulled abstractly related tokens closer together and captured non-literal wordplay that defeated the static pipeline. Though this was still not enough as a few thousand categories cannot cover the possibilities of groupings in natural language.

In contrast, the Large Language Models vastly outperformed both the localized automated methods and the general human average. While the general human baseline solve rate for the game sits at approximately 70 percent, generative reasoning models like Gemini and Claude achieved near-perfect accuracy, consistently navigating deceptive red herrings. This success was not uniform across all generative architectures, however, as ChatGPT struggled significantly and failed to resolve approximately half of the provided test puzzles. The disparity in these results shows that resolving these boards is a contextual reasoning problem rather than a pure clustering problem. When a puzzle introduces categories that rely heavily on cultural knowledge or orthographic tricks, localized clustering algorithms are mathematically blind, whereas advanced LLMs succeed by evaluating the probability of the words functioning as a shared linguistic joke or idiom.

7. Future Work and Limitations

7.1 Siamese Network

The primary shortcoming of the Siamese model is data volume. Generalizing the nuances of natural language requires significantly more than a few hundred clusterings. The model only had access to 915 archived games to map the complex categories. Also, future iterations will require integrating a larger sentence transformer, such as mpnet-base-v2, to establish a wider contextual understanding. Though it is more computationally expensive, the larger sentence transformer has double the vector space compared to MiniLM L6-V2.

7.2 Static Embeddings and Partition Solvers

The fundamental flaw with static embeddings is that they cannot cover all possible word connections. A target category might be so obscure that the included words have never actually been seen together in text, resulting in zero semantic or structural similarity.

Furthermore, the search algorithm presents a distinct bottleneck. While the greedy DFS will find the highest-scoring board in most standard cases, there are edge cases where it incorrectly returns a local minimum on a single category, causing the rest of the board to fail. To solve this, future iterations should implement dedicated partition solvers. Running global partitioning algorithms across the nearly 2.6 million unique groupings would guarantee the mathematically optimal group, circumventing the local minimum trap, though this will require absorbing a significantly higher computational cost.

8. Conclusion

Ultimately, Connections serve as an effective benchmark because it cannot be solved without first addressing the broader difficulties inherent to Natural Language Processing (NLP). Natural language is notoriously difficult to map onto pure vector distances because words frequently exhibit polysemy. Words hold multiple different definitions, varying pronunciations, and shift meaning entirely based on context, all while retaining the exact same spelling. This difficulty is only furthered by the game's intentional inclusion of abstract categories that exist entirely beyond traditional vector semantics. Relying on mechanics like anagrams, fill-in-the-blank phrases, and niche cultural knowledge means the ultimate solution for semantic grouping must blend strict mathematical similarity with dynamic contextual reasoning.